

TP 1 - traitement d'images - partie 1

OBJECTIFS DU TP

Ce TP a pour objectif de manipuler des images numériques avec Python, NumPy et OpenCV.

À l'issue de la séance, vous devrez être capables :

- d'examiner la représentation matricielle d'une image ;
- de calculer et d'interpréter un histogramme ;
- d'expliquer les effets de l'échantillonnage et de la quantification ;
- de manipuler les différents canaux d'une image couleur ;
- d'analyser les dégradations produites par des transformations géométriques successives et par différents modes d'interpolation.

L'objectif n'est pas seulement d'obtenir un programme fonctionnel, mais de relier les résultats observés aux notions de traitement d'images étudiées en cours.

OpenCV est une bibliothèque open source de traitement d'images et de vision par ordinateur. Dans ce TP, elle sera utilisée avec NumPy pour représenter et manipuler les images.

INSTALLATION DE L'ENVIRONNEMENT DE TRAVAIL

Au début de la séance, vous allez créer un environnement Python dédié au TP et installer les bibliothèques nécessaires.

CRÉATION DE L'ENVIRONNEMENT PYTHON

Vous pouvez utiliser la méthode et l'environnement de votre choix (conda, venv, etc.).

La procédure suivante donne un exemple avec conda. Elle suppose que Conda ou Miniconda est déjà installé sur la machine.

Dans un terminal, placez-vous dans le répertoire du TP, puis créez un environnement nommé `vpo` :

```
conda create -n vpo python pip
```

Activez ensuite cet environnement :

```
conda activate vpo
```

INSTALLATION DE OPENCV

Installez la version complète d'OpenCV, comprenant les modules supplémentaires du dépôt `opencv_contrib` :

```
python -m pip install --upgrade pip
python -m pip install opencv-contrib-python numpy matplotlib
```

VÉRIFICATION DE L'INSTALLATION

Vérifiez enfin l'installation :

```
python -c "import cv2; print(cv2.__version__)"
```

La dernière commande doit afficher le numéro de version d'OpenCV sans produire de message d'erreur.

Enfin, exécutez le programme Python fourni avec le TP. Celui-ci doit charger et afficher l'image `monarch.png`. En cas d'erreur, vérifiez que l'environnement `vpo` est toujours activé et que le chemin de l'image est correct.

RENDUS ATTENDUS

Pour ce TP, vous devez rendre :

- un compte rendu au format **PDF** présentant les résultats demandés et leur interprétation ;
- les scripts Python réalisés pour obtenir les différents traitements.

Le compte rendu ne doit pas être une succession de captures d'écran. Sélectionnez les comparaisons qui permettent de répondre aux questions et indiquez les paramètres utilisés.

UTILISATION DE L'IA GÉNÉRATIVE

L'utilisation d'une IA générative est autorisée pour rechercher une fonction, expliquer une erreur ou proposer une première version de code. Vous restez responsables du programme rendu et devez être capables d'en expliquer le fonctionnement et de justifier les choix effectués.

Dans le compte rendu, indiquez brièvement les usages significatifs d'une IA générative et présentez au moins un élément de code ou une explication que vous avez vérifié, corrigé ou amélioré.

1 PRÉSENTATION DE LA STRUCTURE DE DONNÉES IMAGE

On se propose dans un premier temps de faire un redimensionnement "à la main" d'une image en OpenCV. Si en C++ le format utilisé par OpenCV pour gérer les images est `cv::Mat`, en Python une image est automatiquement convertie en un objet de type `numpy.ndarray`. Autrement dit, OpenCV utilise NumPy pour représenter et manipuler les images.

Chaque matrice possède un nombre de lignes (`rows`) et de colonnes (`cols`). Le nombre de lignes représente la hauteur de l'image alors que le nombre de colonnes représente la largeur et le pixel (0,0) est en haut à gauche.

Selon le type d'image :

- Une image en niveaux de gris est une matrice 2D (hauteur x largeur) avec des entiers entre 0 (noir) et 255 (blanc).
- Une image couleur est une matrice 3D (hauteur x largeur x 3) représentant les 3 canaux de couleur : Bleu, Vert, Rouge (BGR).

Attention : Contrairement à la norme RGB, la bibliothèque OpenCV utilise le format BGR (Bleu, Vert, Rouge) pour représenter les images couleur.

La forme du tableau numpy pour coder l'image va dépendre de la profondeur de l'image (niveau de gris ou couleur) et du nombre de canaux :

- Chaque pixel d'une image en niveaux de gris est une valeur unique, soit des valeurs de 0 à 255 par canal.
- Chaque pixel d'une image couleur est un triplet BGR.
- Chaque pixel d'une image avec canal alpha (transparence) possède 4 valeurs (B, G, R, A)

```
import cv2
import numpy as np

# Chargement en niveaux de gris
img_gray = cv2.imread('monarch.png', cv2.IMREAD_GRAYSCALE)
print(f"Forme (gris) : {img_gray.shape}, Type : {img_gray.dtype}")
# Forme typique : (H, W)

# Chargement en couleur (BGR)
img_color = cv2.imread('monarch.png', cv2.IMREAD_COLOR)
print(f"Forme (couleur) : {img_color.shape}, Type : {img_color.dtype}")
# Forme typique : (H, W, 3)

# Chargement d'une image avec canal alpha (par exemple PNG avec transparence)
img_rgba = cv2.imread('image_alpha.png', cv2.IMREAD_UNCHANGED)

print(f"Forme : {img_rgba.shape}") # (H, W, 4)
```

```
print(f"Type : {img_rgba.dtype}")

# Accès à un pixel (100, 150) :
b, g, r, a = img_rgba[100, 150]
print(f"B: {b}, G: {g}, R: {r}, A: {a}")
```

Si vous souhaitez un peu plus d'exemples, vous pouvez aller à l'adresse suivante :
https://docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html

Le code OpenCV fourni sur la plateforme pédagogique pour le TP vous permet d'ouvrir une image en niveau de gris (par défaut ../imagesDeTest/monarch.png ou celle dont le chemin d'accès est passé en paramètre lors de l'exécution) et de l'afficher dans une fenêtre tant que la touche **ESC** n'est pas enfoncée. Commencez par exécuter ce code si ce n'est pas déjà fait.

VÉRIFICATION

Exécutez le programme précédent et vérifiez que les informations affichées sont cohérentes avec les images chargées.

Avant de poursuivre le TP, assurez-vous de savoir :

- interpréter les différentes dimensions de `image.shape` ;
- identifier le type des pixels avec `image.dtype` ;
- accéder à un pixel avec la notation `image[y, x]` ;
- interpréter les composantes d'un pixel couleur stocké par OpenCV.

Aucun résultat n'est demandé dans le compte rendu pour cette partie.

2 HISTOGRAMME

L'histogramme d'une image en niveaux de gris indique, pour chaque niveau compris entre 0 et 255, le nombre de pixels possédant cette valeur.

2.1 HISTOGRAMME DE L'IMAGE ORIGINALE

Chargez l'image `peppers-512.png` en niveaux de gris. Utilisez la fonction `cv2.calcHist` pour calculer son histogramme, puis affichez celui-ci avec `matplotlib`.

Lors de l'affichage, l'axe horizontal doit représenter les niveaux de gris de 0 à 255 et l'axe vertical le nombre de pixels.

Conservez le code permettant de calculer et d'afficher un histogramme : il sera réutilisé pour comparer les différentes images produites dans la suite.

2.2 ÉGALISATION DE L'HISTOGRAMME

Appliquez une égalisation à l'image avec la fonction `cv2.equalizeHist`.

Affichez l'image obtenue et son histogramme. Comparez-les à l'image originale et à son histogramme.

2.3 MODIFICATION DE LA LUMINOSITÉ ET DU CONTRASTE

Consultez le tutoriel OpenCV suivant :

https://docs.opencv.org/4.x/d3/dc1/tutorial_basic_linear_transform.html

La modification de la luminosité et du contraste peut être réalisée avec la fonction `cv2.convertScaleAbs` :

```
resultat = cv2.convertScaleAbs(image, alpha=alpha, beta=beta)
```

Le paramètre `alpha` agit principalement sur le contraste, tandis que le paramètre `beta` agit principalement sur la luminosité.

À partir de l'image originale, réalisez les transformations suivantes :

1. augmentez la luminosité avec `alpha=1` et `beta=40` ;
2. augmentez le contraste avec `alpha=1.5` et `beta=0`.

Pour chaque transformation, affichez l'image obtenue et son histogramme. Comparez-les avec l'image originale et son histogramme. Commentez les modifications observées.

COMPTE RENDU

Présentez les images et les histogrammes permettant de comparer :

- l'image originale ;
- l'image après égalisation ;
- l'image après augmentation de la luminosité ;
- l'image après augmentation du contraste.

Organisez ces résultats en une ou plusieurs figures comparatives plutôt qu'en une succession de captures d'écran. Pour chaque transformation, indiquez les paramètres utilisés.

Interprétez les résultats en répondant aux questions suivantes :

1. Comment l'égalisation modifie-t-elle la répartition des niveaux de gris et la perception du contraste ?
2. Quel effet l'augmentation de `beta` produit-elle sur l'image et sur son histogramme ?
3. Quel effet l'augmentation de `alpha` produit-elle sur l'image et sur l'étalement de son histogramme ?
4. Parmi les transformations réalisées, laquelle améliore le plus la lisibilité de l'image étudiée ? Justifiez votre réponse à partir de vos observations.

3 REDIMENSIONNEMENT D'IMAGE

Le redimensionnement modifie le nombre de pixels d'une image. Une réduction peut supprimer des informations, tandis qu'un agrandissement doit créer de nouvelles valeurs à partir des pixels existants.

Le tutoriel OpenCV suivant présente les principales fonctions utilisées pour les transformations géométriques : https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html

3.1 SOUS-ÉCHANTILLONNAGE

Chargez l'image `peppers-256.png` en niveaux de gris. On souhaite obtenir une image de dimensions 64×64 en conservant un pixel par bloc de 4×4 pixels.

Écrivez une fonction respectant l'interface suivante :

```
def sous_echantillonner(image, facteur):  
    # À compléter
```

Cette première version doit construire la nouvelle image à l'aide d'une double boucle. **Vous ne devez pas utiliser `cv2.resize` dans cette fonction.**

Vérifiez que l'image obtenue possède bien les dimensions attendues.

Réalisez ensuite la même réduction avec `cv2.resize`, en utilisant successivement :

- l'interpolation au plus proche voisin : `cv2.INTER_NEAREST` ;
- l'interpolation par aire : `cv2.INTER_AREA`.

Remarque : La méthode `cv2.INTER_AREA` calcule chaque pixel de l'image réduite à partir des pixels de la zone correspondante dans l'image originale. Elle limite ainsi la perte brutale d'information produite par la sélection d'un seul pixel et est particulièrement adaptée à la réduction d'images.

Comparez les images obtenues avec les trois méthodes.

3.2 SUR-ÉCHANTILLONNAGE

Chargez l'image `peppers-128.png` en niveaux de gris, puis agrandissez-la aux dimensions 512×512 avec `cv2.resize`.

Utilisez successivement :

- l'interpolation au plus proche voisin : `cv2.INTER_NEAREST` ;
- l'interpolation bilinéaire : `cv2.INTER_LINEAR`.

Affichez les deux images obtenues à la même échelle et observez en particulier les contours et les zones présentant de fortes variations d'intensité.

COMPTE RENDU

Présentez une comparaison des résultats obtenus lors de la réduction et de l'agrandissement. Indiquez pour chaque image la méthode utilisée.

Interprétez vos résultats en répondant aux questions suivantes :

1. Pourquoi le passage d'une image 256×256 à une image 64×64 conduit-il à conserver un pixel par bloc de 4×4 ?
2. Comparez le résultat obtenu en sélectionnant un seul pixel par bloc de 4×4 avec celui obtenu par `cv2.INTER_AREA`. Quelles différences observez-vous, notamment dans les contours et les zones contenant de petits détails ?
3. Lors de l'agrandissement, quelles différences observez-vous entre l'interpolation au plus proche voisin et l'interpolation bilinéaire ?
4. L'agrandissement permet-il de retrouver des détails absents de l'image 128×128 ? Justifiez votre réponse.

4 QUANTIFICATION, LUT ET PLANS DE COULEUR

4.1 QUANTIFICATION DES NIVEAUX DE GRIS

La quantification consiste à réduire le nombre de niveaux utilisés pour représenter une image. Chargez l'image `peppers-512.png` en niveaux de gris, puis produisez des images quantifiées sur :

64, 16, 4 et 2 niveaux de gris.

Pour une quantification sur N niveaux, vous pouvez calculer le pas de quantification avec :

`pas = 256 // N`

Utilisez ensuite des opérations sur les tableaux NumPy pour remplacer chaque pixel par le niveau quantifié correspondant. N'utilisez pas de boucle parcourant individuellement tous les pixels.

Affichez les images quantifiées et leurs histogrammes.

4.2 APPLICATION DE LUT COLORÉES

Une table de correspondance, ou LUT (*Look-Up Table*), associe à chaque niveau de gris une couleur d'affichage. Elle permet de produire une représentation en fausses couleurs sans modifier les structures présentes dans l'image.

À partir de l'image originale en niveaux de gris, appliquez les quatre cartes de couleurs suivantes avec `cv2.applyColorMap` :

- `cv2.COLORMAP_JET` ;
- `cv2.COLORMAP_HOT` ;
- `cv2.COLORMAP_OCEAN` ;
- `cv2.COLORMAP_PINK`.

Affichez l'image originale et les quatre représentations colorées.

COMPTE RENDU

Présentez :

- l'image originale et les images quantifiées ;
- les histogrammes permettant de visualiser la réduction du nombre de niveaux ;
- une comparaison de l'image en niveaux de gris avec les quatre LUT colorées.

Interprétez les résultats en répondant aux questions suivantes :

1. Comment la diminution du nombre de niveaux se manifeste-t-elle dans l'image et dans son histogramme ?
2. Pour quel nombre de niveaux les transitions entre régions deviennent-elles nettement visibles dans l'image étudiée ?
3. L'application d'une LUT colorée ajoute-t-elle de nouvelles informations à l'image ou modifie-t-elle seulement leur représentation ? Justifiez votre réponse.

4. Comparez les représentations obtenues avec les quatre LUT. Quelles différences observez-vous dans la mise en évidence des faibles et des fortes intensités ? Quelle LUT vous semble la plus adaptée à l'image étudiée ? Justifiez votre réponse.

4.3 SÉPARATION DES PLANS DE COULEUR

Chargez l'image `peppers-512-RGB.png` en couleur. OpenCV stocke les canaux dans l'ordre BGR. Séparez-les avec :

```
b, g, r = cv2.split(image)
```

Convertissez également l'image originale en niveaux de gris :

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Affichez dans une même figure :

- l'image couleur originale ;
- le canal bleu ;
- le canal vert ;
- le canal rouge ;
- l'image convertie en niveaux de gris.

Lors de l'affichage de l'image couleur avec `matplotlib`, pensez à convertir son ordre de couleurs de BGR vers RGB.

La conversion en niveaux de gris réalisée par OpenCV tient compte différemment des trois composantes. Elle peut être approximée par :

$$Y = 0.299R + 0.587G + 0.114B.$$

COMPTE RENDU

Présentez une figure comparative regroupant l'image originale, les trois plans de couleur et l'image en niveaux de gris.

Interprétez les résultats en répondant aux questions suivantes :

1. Comment la couleur des différentes régions de l'image influence-t-elle leur intensité dans les plans rouge, vert et bleu ?
2. D'après les coefficients utilisés pour la conversion en niveaux de gris, quel canal contribue le plus à la luminosité obtenue et quel canal contribue le moins ?
3. Identifiez dans l'image une région où le résultat en niveaux de gris diffère nettement du plan vert. Expliquez cette différence à partir des valeurs des autres canaux.

5 ROTATION

Chargez l'image `squares.tif`. Vous allez comparer une rotation unique avec plusieurs rotations successives conduisant au même angle total.

Pour réaliser une rotation, utilisez :

- `cv2.getRotationMatrix2D` pour construire la matrice de transformation ;
- `cv2.warpAffine` pour appliquer la transformation à l'image.

Un exemple est disponible dans le tutoriel OpenCV suivant : https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html

Le centre de rotation doit être placé au centre de l'image et les dimensions de l'image de sortie doivent rester identiques à celles de l'image originale.

Remarque : avec OpenCV, un angle positif correspond à une rotation dans le sens anti-horaire. Pour effectuer une rotation horaire, utilisez donc un angle négatif.

5.1 INTERPOLATION AU PLUS PROCHE VOISIN

Utilisez l'interpolation `cv2.INTER_NEAREST` pour produire :

1. une rotation directe de 90° dans le sens horaire ;
2. deux rotations successives de 45° dans le sens horaire ;
3. six rotations successives de 15° dans le sens horaire.

Pour les rotations successives, chaque rotation doit être appliquée au résultat de la rotation précédente.

5.2 INTERPOLATION BILINÉAIRE

Recommencez les trois expériences précédentes en utilisant l'interpolation bilinéaire `cv2.INTER_LINEAR`.

COMPTE RENDU

Regroupez les six images obtenues dans une même figure comparative : une ligne pour l'interpolation au plus proche voisin et une ligne pour l'interpolation bilinéaire. Indiquez clairement le nombre et l'angle des rotations appliquées.

Interprétez les résultats en répondant aux questions suivantes :

1. Les trois procédures conduisent-elles exactement au même résultat alors que l'angle total est toujours de 90° ? Expliquez les différences observées.
2. Quel effet le nombre de rotations successives produit-il sur la qualité de l'image ?
3. Comparez les défauts produits par l'interpolation au plus proche voisin et par l'interpolation bilinéaire.
4. Observez les bords de l'image après rotation. Quelle information peut être perdue lorsque les dimensions de l'image de sortie restent inchangées ?